Selenium Interview Q&A – Part 2

1. What happens if you receive browser notification during test automation Execution?

If browser notifications appear during test execution, they can disrupt the flow of automation and cause failures if not handled properly.

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;

// Set up ChromeOptions to block notifications
ChromeOptions options = new ChromeOptions();
Options.addArguments("--disable-notifications");
```

A. Explanation:

- Browser notifications, such as **location** acce**s**s or **message alerts**, can overlay web elements, preventing Selenium from interacting with them. This leads to exceptions like **ElementNotInteractableException**.
- Most browsers allow blocking notifications by modifying browser-specific settings via Options classes (Ex: ChromeOptions)
- For Chrome, we disable notifications using the **argument --disable-notifications**
- **ChromeOptions Class --** Used to customize browser settings before launching Chrome.
- **Add Arguments --** options.addArguments("--disable-notifications"); prevents browser notifications from showing up.
- **Passing Options to WebDriver --** new ChromeDriver(options) initializes the browser with the specified settings

2. What steps will you take if an element is not interactable in selenium?

To handle an element not being interactable, I would take a series of steps to identify and fix the issue based on the cause.

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

WebElement getStarted = driver.findElement(By.id("GetStarted"));

// Step 1: Wait until the element is visible

WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
Wait.until(ExpectedConditions.visibilityOf(getStarted);

// Step 2: Scroll to the element
JavaScriptExecutor js = (JavaScriptExecutor) driver;
js.executeScript("arguments[0].scrollIntoView(true);", getStarted);

//Step 3: Ensure the element is clickable
Wait.until(ExpectedConditions.elementToBeClickable(getStarted));
```

B. Explanation:

- **Root Causes of the Issue –** If the element is not interactable:
  - It might be **hidden by Css** or **outside the viewpoint**
  - It might be overlapped by another element (**like a pop-up** or **loader**)
  - It might not ready for interaction (**Ex: still loading** or **disabled**)
- Use explicit waits **ExpectedConditions.visibilityOfElementLocated()** to ensure the element is visible in the DOM and on the screen.
- If the element is outside the viewport, use JavaScript to scroll to it. The script **arguments[0].scrollIntoView(true);** ensures the element comes into focus.
- Use **ExpectedConditions.elementToBeClickable()** to confirm the element is enabled and ready for interaction.
- Retrieve all window handles using **driver.getWindowHandles(),** which returns a set of all window identifiers.

3. How will you handle disappearing elements in selenium

To handle disappearing elements, I would write an **XPath** that dynamically identifies elements based on their attributes or parent-child relationships, ensuring robustness.

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;


// Using dynamic XPath for a disappearing element
String dynamicXpath = "//div[contains(@class, 'message') and contains (text(), Success")];


// Using explicit wait to locate the disappearing element
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
Wait.until(ExpectedConditions.presenceOfElementLocatedBy(By.xpath(dynamicXpath)));
```

C. Explanation:

- Disappearing elements may appear for a short duration or load dynamically.
- Using **WebDriverWait** with **ExpectedConditions.presenceOfElementLocated** synchronizes the script with the element's appearance in the DOM

- Identify elements with unique or consistent attributes
  - **Example:** //div[contains(@class, 'message')] finds all div tags with a class containing "message"
- Target elements by partial or exact text matching.
  - **Example:** //div[contains(text(), 'Success')] matches div tags containing the word "Success"
- Use axes like **ancestor**, **descendant**, or **following-sibling** to locate elements relative to others.
- **TakesScreenshot —** Allows capturing screenshots in Selenium

(Might have the question) For Beginners:

- **Here we have not used driver.findElement**: Because it searches the element immediately, if the element is not present in the DOM or it is not interactable at that specific moment, Selenium throws a **NoSuchElementException**.

4. How to check presence of an element on webpage using in selenium?

To check the presence of an element on a webpage, we can use **findElements** and verify if the size of the returned list is greater than 0.

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;


// Check if the element is present
boolean isElementPresent = driver.findElement(By.cssSelector("Button")).size() > 0;

if(isElementPresent) {
      System.out.println("Element is present on the webpage");
} else {
      System.out.println("Element is not present on the webpage");
}
```

A. Explanation:

- **findElement** throws a **NoSuchElementException** if the element is not found, which interrupts the script.
- **findElements** returns a list of matching elements. If no elements are found, it returns an empty list, allowing the script to handle the absence gracefully

5. How will you handle Calendars in selenium?

To handle calendars in Selenium, I would locate the calendar element and interact with its components (**Ex: date**, **month**, or **year**) using appropriate locators and logic.

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;

// Open the calender
WebElement calendarIcon = driver.findElement(By.id("calendaricon"));
calendarIcon.click();

// Select a specific date (Ex: 15th of the current month)
WebElement date = driver.findElement(By.id("date"));
date.click();
```

A. Explanation:

- Calendars are usually implemented as custom widgets on web pages, with dates displayed in a grid format.
- Handling them involves locating elements dynamically based on the desired date, month, or year.
- Use a unique locator to find the calendar icon or input field and click it to open the calendar. **Ex: driver.findElement(By.id("calendarIcon")).click()**
- Locate the specific date using an XPath, such as **//td[text()='15']** which selects the table cell containing the date "15"
- If the calendar spans multiple months or years, write logic to click "**Next**" or "**Previous**" buttons until the desired month/year is displayed

```
while (!driver.findElement(By.className("Month")).getText().contains("December")) {
    driver.findElement(By.xpath("//button[text()='Next']")).click();
}
```

6. What are implicit & Explicit wait in selenium?

Implicit waits set a default wait time for locating elements, while **explicit waits** define conditions for specific elements before proceeding.

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.interactions.Actions;

// Set implicit wait
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

driver.get("https://activetheory.net/");

WebElement element = driver.findElement(By.id("elementId"));

// set explicit wait
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
Wait.until(ExpectedConditions.visibilityOf(element));
```

A. Explanation:

- **Implicit Wait**
  - Sets a default waiting time for the WebDriver to check if elements are present in the DOM. It applies to all **findElement** and **findElements** calls.
  - If the element is found before the time expires, the script proceeds immediately.
  - Best for static pages where all elements load within a predictable time frame.
- **Explicit Wait**
  - Waits for specific conditions (like **visibility**, **clickability**) of a specific element before proceeding.
  - Allows finer control and is element-specific
  - Ideal for dynamic pages where elements load asynchronously or depend on user actions.

7. What is tag & roles of tags in TestNG?

In TestNG, tags are used to group or categorize test methods. They help organize tests into specific groups or suites, making it easier to manage large test suites.

The main types of tags in TestNG are **@Test**, **@BeforeTest**, **@AfterTest**, **@BeforeMethod**, **@AfterMethod**, and others for controlling execution flow.

```
@Test
public void testLogin() {
      // Login logic
}

@Test
public void testSignUp() {
      // SignUp logic
}
```

- The **@Test** annotation in TestNG is used to mark a method as a **test method** or **test Case**.
- This tells TestNG to treat the method as a test, which will be executed as part of the testing process.
- Without this annotation, TestNG would not recognize the method as a part of the test suite.
- The test methods marked with **@Test** are executed in the order defined by their priority or alphabetically if no priority is specified. They can also be grouped or depend on other methods.
- **@Test(enabled = false) –** Allow us to disable a test method (It will not execute the methos)
- **@Test(priority = 1) --** Specifies the order of execution for multiple test methods. Methods with lower priority values are executed first
- **@Test(dependsOnMethods = {"testLogin"} --** Defines dependencies between test methods, ensuring that certain tests are only run if others pass.
  - **Example:** testSignup() runs only if testLogin() passes.

```
@BeforeTest
public void setUp() {
	// Setup logic before tests
}
```

- When TestNG detects a method annotated with **@BeforeTest**, it will execute that method once before any of the test methods (annotated with @Test) in the class or suite are executed.
- We can use **@BeforeTest** for any setup that needs to be done before the tests run. This can include actions like:
  - Opening the browser | Navigating to a webpage | Setting up test data.
  - Logging into the application

Real-time Example:

In Songdew (My Project), before starting any tests related to song uploads, I would use **@BeforeTest to open the browser**, **log into the user account**, and **navigate to the music upload page**. This way, all test methods related to the upload functionality can start from the same initial setup state.

```
@AfterTest
public void cleanUp() {
	// Setup logic after tests
}
```

- The **@AfterTest** annotation in TestNG marks a method that should be executed after all the **@Test** methods in the current test class or suite have been executed.
- This is typically used for final cleanup tasks that need to be done after all tests are complete.
- We can use **@AfterTest** to release any system resources, such as **closing database connections**, **stopping services**, or **clearing logs**, after all tests have run.

Real-time Example:

In Songdew (My Project) after running all tests related to the music upload and sharing features, we might use **@AfterTest** to stop the server or database connection that was used for the test. This sures that resources are properly cleaned up once testing is complete.

```
@BeforeMethod
public void beforeEachTest() {
      // Setup logic before each tests
}
```

- TestNG executes the method annotated with **@BeforeMethod** before each test method marked with **@Test**. This ensures that any setup is done fresh for each test.
- We can use **@BeforeMethod** to perform setup tasks that are required before every test, such as opening a new browser window, navigating to a URL, or logging into the application.

Real-time Example:

In Songdew (My Project) if we are testing various functionalities (Ex: song upload, profile editing), we could use **@BeforeMethod** to ensure that the browser is opened and logged in before each test. This helps maintain consistency in the test environment.

```
@AfterMethod
public void beforeEachTest() {
      // Setup logic after each tests
}
```

- TestNG ensures that methods annotated with **@AfterMethod** are executed automatically after each **@Test** method, allowing for efficient cleanup.
- It's commonly used for cleanup operations that need to be done after each test, such as **closing browsers** or **clearing test data**
- After each test method runs, we can use **@AfterMethod** to clean up actions like:
  - Closing the browser | logging out of the application | Resetting test data.

Real-time Example:

In Songdew (My Project) after performing a test like uploading a song or updating a profile, we may want to use **@AfterMethod** to **log out of the application** and **close the browser**. This ensures that the next test starts from a clean state without any leftover session data.

```
@BeforeClass
public void setUpBeforeClass() {
// Perform setup actions like opening the browser,
      initializing configurations, etc.
}

@AfterClass
public void tearDownAfterClass() {
// Perform cleanup actions like closing the browser, stopping services, etc.
}
```

- TestNG will execute the **@BeforeClass** method once, before the test methods are executed. This ensures that any setup required for the entire test class is done before running the actual tests.
- It is ideal for setup tasks that only need to be done once, such as opening a browser or initializing a test environment for the whole class
- TestNG will execute the **@AfterClass** method once, after the test methods have completed. This ensures that any global cleanup or resource release happens only once for the whole test class.

Real-time Example:

In Songdew (My Project) before running all the tests in a class, we could use **@BeforeClass** to **open the browser** and **log in to the application**. Then, after all tests are completed, we could use **@AfterClass** to **log out** and **close the browser**.

```
@DataProvider(name = "loginData")
public object[][] loginData() {
      return new Object[][] {
            {"user1", "password1"},
            {"user2", "password2"},
            {"user3", "password3"},
      };
}

@Test(dataProvider = "loginData")
public void testLogin(String username, String password) {
      // Perform login actions here
}
```

- The **@DataProvider** annotation allows a method to provide test data to another method. The test method uses the data passed from **@DataProvider** and executes the same test with different inputs.
- TestNG will automatically invoke the test method multiple times, each time with a different set of data provided by the **@DataProvider** method.
- When you need to test the same logic with different data sets **(Ex: testing login with various username and password combinations)**

```
public class TestNGFactoryExample {

	private String username;
	private String password;


	// Constructor to pass parameters to the test
	public TestNGFactoryExample(String username, String password) {
		this.username = username;
		this.password = password;
	}

@Factory
public object[] createTests() {
	return new Object[] {
		new TestNGFactoryExample("user1", "password1"),
		new TestNGFactoryExample("user2", "password2"),
		new TestNGFactoryExample("user3", "password3"),
	};
}
```

- The **@Factory** annotation is used to create instances of test classes dynamically. It allows you to pass parameters to constructors to vary the behavior of test methods for different scenarios.
- TestNG will execute the test class multiple times, creating new instances of the class for each set of data returned by the @Factory method.
- When we need to run the same test class with different parameters or configurations.
- Suitable for tests where each instance of the test requires different constructor arguments.